Failure (search fails mid-run) — Hints

> Back to the exercise · Hint 1 · Hint 2 · Solution

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 2 depth levels: **Basic** (the plain idea) and **Intermediate** (proper Python, plus the cases a real project has to handle).

- Hint 1 — The idea, and the exact pieces
- Hint 2 — The plan, and almost the whole thing

Back to the exercise · Hint 1 · Hint 2 · Solution

Hint 1 — The idea, and the exact pieces

Basic Version

Make a two-node graph — `plan_node`, then `search_node` — with a checkpointer, and make `search_node` raise on purpose. Catch the error, then call `graph.get_state(config)`: `plan_node`'s work should still be saved, and `.next` should say the run stopped before `search_node`.

Back to the exercise · Hint 1 · Hint 2 · Solution

Intermediate Version

The exact pieces:

- `try: graph.invoke(...)` / `except RuntimeError` — and a `raise AssertionError(...)` right after `invoke()`, so the test fails if the crash ever stops happening.
- `snapshot = graph.get_state(config)` — `.values` is the saved state, `.next` the node that was about to run (Doc09).
- `build_graph(search_node)` taking the search node as a parameter, so later approaches plug in a real one.

Then the real question: should every failure fail the run? A `TimeoutError` or `ConnectionError` is often gone a second later — retry it, waiting a little longer each time (Doc02's backoff). A bad query or a bug fails the same way every time — raise it at once. To *test* that, you need a search that fails on demand: Doc07's fail-once counter idea, as a small `FlakySearch` class.

Back to the exercise · Hint 1 · Hint 2 · Solution

Hint 2 — The plan, and almost the whole thing

Basic Version

```
plan_node:   {"plan": "search the documents for: " + task}
search_node: raise RuntimeError("simulated vector store outage")
graph: START -> plan -> search -> END, MemorySaver()

try invoke, catch the RuntimeError,
then print graph.get_state(config).values and .next
```

> Back to the exercise · Hint 1 · Hint 2 · Solution

Intermediate Version

```
TRANSIENT_ERRORS = (TimeoutError, ConnectionError); MAX_ATTEMPTS = 3

search_with_retry(search, query):
    for attempt in range(MAX_ATTEMPTS):
        try: return search(query)
        except TRANSIENT_ERRORS: last attempt -> raise; else sleep 2 ** attempt

FlakySearch(fail_times, error): .search(query) raises `error` for the
    first fail_times calls (count them in call_count), then really searches

tests: 2 timeouts then success (call_count == 3); a ValueError raised at
       once (call_count == 1); retries used up -> state still saved
```

Difference between Basic and Intermediate: Basic proves state survives one crash. Intermediate retries only the errors worth retrying, and proves every outcome with a search that fails on demand.

Back to the exercise · Hint 1 · Hint 2 · Solution

Full solution: Show me the solution